Programación

Nombre: _____ Fecha: _____

Qué vas a aprender

- Representar un algoritmo con un diagrama de flujo
- Usar variables, condicionales, bucles y funciones
- Programar una placa microcontroladora para leer sensores y controlar actuadores
- Usar bucles, condiciones compuestas y operadores en programas
- Probar y depurar programas que controlan una placa con sensores y actuadores

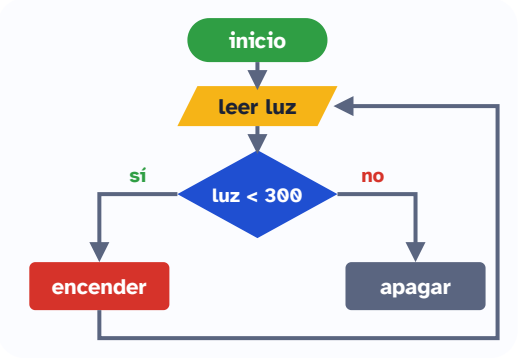
Para pensar

Las farolas de tu calle se encienden solas al anochecer. Si tuvieras que darles las instrucciones, ¿qué les dirías?

Explicación

Algoritmos y diagramas de flujo

Un **algoritmo** es una secuencia ordenada y finita de pasos para resolver un problema. Antes de programar conviene dibujarlo con un **diagrama de flujo**: óvalos para el **inicio** y el **fin**, rectángulos para las **acciones**, rombos para las decisiones (sí/no) y paralelogramos para la **entrada y salida** de datos. Las flechas indican el orden.



Variables y tipos de datos

Una **variable** es una «caja» con nombre donde el programa guarda un dato que puede cambiar: la lectura de un sensor, un contador, una puntuación. Cada variable tiene un **tipo**: número entero (`int`), decimal (`float`), verdadero/falso (`bool`) o texto (`String`). Con ellas se hacen **operaciones** aritméticas (+, –, *, /) y **comparaciones** (<, >, ==, !=).



Operadores

Los programas calculan y comparan con **operadores**. Los **aritméticos** hacen cuentas: + (suma), – (resta), * (multiplicación), / (división) y % (resto de la división;

7 % 3 da 1, útil para saber si un número es par). Los de **comparación** dan verdadero o falso: == (igual), != (distinto), <, >, <= y >=. Ojo: un solo = asigna un valor a una variable (x = 5), y dos == comparan (¿x == 5?). Los **lógicos** combinan condiciones: **Y** (&&, las dos deben cumplirse), **O** (||, basta una) y **NO** (!, invierte). Por ejemplo: «si hay poca luz Y hay alguien en la sala, enciende la lámpara».

Operador	Significado	Ejemplo	Resultado
%	resto	10 % 4	2
==	igual a	3 == 3	verdadero
!=	distinto de	3 != 5	verdadero
&&	y	(5 > 3) && (2 > 4)	falso
||	o	(5 > 3) || (2 > 4)	verdadero

undefined

Estructuras de control

Los programas no siempre van en línea recta. El **condicional** `if … else` ejecuta unas instrucciones solo si se cumple una condición. Los **bucles** repiten instrucciones: `for` un número fijo de veces (parpadear 10 veces) y `while` mientras se cumpla una condición (esperar a que se pulse un botón). En Arduino, la función `loop()` ya es un bucle infinito.

```
for (int i = 0; i < 10; i++) {
  digitalWrite(13, HIGH);
  delay(500);
  digitalWrite(13, LOW);
  delay(500);
}
```

repite 10 veces: enciende, espera, apaga

Los bucles en detalle

Un bucle **for** se usa cuando sabemos cuántas veces repetir. Tiene tres partes: un valor inicial del contador, una condición para seguir y un incremento. `for (int i = 0; i < 5; i++)` repite 5 veces, con i valiendo 0, 1, 2, 3 y 4. Un bucle

while («mientras») repite mientras se cumpla una condición, sin saber de antemano cuántas veces: «mientras no se pulse el botón, haz parpadear el LED». Hay que asegurarse de que la condición acabe siendo falsa; si no, el programa queda atrapado en un **bucle infinito**. En Arduino, la función loop() es en sí misma un bucle infinito que repite el programa principal sin parar.

undefined

Funciones

Una **función** es un bloque de código con nombre que hace una tarea concreta y se puede llamar muchas veces. Así el programa queda más corto, ordenado y fácil de corregir. Pueden recibir **parámetros** (datos de entrada) y devolver un resultado. Por ejemplo, `parpadear(3)` podría encender y apagar un LED tres veces. Todo programa de Arduino tiene dos funciones obligatorias: `setup()`, que se ejecuta una vez al empezar, y `loop()`, que se repite sin fin.

```
void setup() {                1 vez
  pinMode(13, OUTPUT);
}
void loop() {                 siempre
  parpadear(3);
  delay(2000);
}
```

Programar una placa: entradas y salidas

Una placa como **Arduino** o **micro:bit** tiene **pines** para conectar sensores y actuadores. Con `digitalRead()` se lee un pulsador (`0` o `1`) y con `analogRead()` un sensor analógico, que da un valor entre `0` y `1023`. Con `digitalWrite()` se enciende o apaga un LED y con `analogWrite()` se regula su brillo o la velocidad de un motor. Se puede programar con **bloques** (mBlock, MakeCode) o con **texto** (lenguaje C++ de Arduino, Python).

```ino
int luz = analogRead(A0);
if (luz < 300) {
  digitalWrite(13, HIGH);
} else {
  digitalWrite(13, LOW);
}
```

LDR en A0 · LED en el pin 13

la farola automática en 6 líneas

Del programa a la placa

Para programar una placa como Arduino se escribe el código en un entorno de programación (el **IDE** de Arduino) o con bloques (mBlock, Arduinoblocks, Tinkercad). El programa se **compila**, es decir, se traduce al lenguaje que entiende el microcontrolador, y se **sube** a la placa por el cable USB. Todo programa tiene dos partes: **setup()**, que se ejecuta una vez al principio para configurar los pines como entradas o salidas, y **loop()**, que se repite sin fin. Para ver qué está pasando dentro se usa el monitor serie: con `Serial.println(valor)` la placa envía al ordenador el valor de una variable, lo que ayuda mucho a encontrar errores. Antes de montar, conviene simular el circuito en Tinkercad Circuits.

undefined

Probar y depurar

Un programa casi nunca funciona a la primera. Los errores de **sintaxis** (un punto y coma olvidado, una llave sin cerrar, una mayúscula mal puesta: `digitalwrite` en vez de `digitalWrite`) los detecta el compilador y no deja subir el programa. Los errores de **lógica** son más difíciles: el programa funciona pero hace otra cosa, por ejemplo porque usamos > en vez de < o el pin equivocado. Para encontrarlos: probar con casos sencillos, mostrar las variables en el monitor serie, comprobar el circuito (muchos fallos son cables mal conectados o un LED al revés) y cambiar una sola cosa cada vez. Escribir **comentarios** (// así) en el código ayuda a entenderlo después.

undefined

Las piezas de cualquier programa

Variables	Condicional	Bucle	Función
guardan datos	decide	repite	agrupa tareas
`int luz = 512;`	`if (luz < 300)`	`for · while · loop`	`parpadear(3);`

Ejercicios

1. **¿Qué tipo de dato usarías para guardar: el número de vidas, la temperatura con decimales, si la puerta está abierta y el nombre del jugador?**

2. **Calcula: 17 % 5, 20 % 4 y 9 / 2 con enteros.**

3. **¿Es verdadera o falsa (x > 3) && (x < 10) si x = 12?**

4. **¿Cuántas veces se repite for (int i = 0; i < 8; i++)?**

5. **Escribe un bucle for que muestre los números del 10 al 1 hacia atrás.**

6. **¿Qué diferencia hay entre for y while?**

7. **Explica qué hacen setup() y loop() en Arduino.**

8. **Escribe una función que reciba un número y devuelva su doble.**

9. **Un LED no se enciende aunque el programa compila. Indica tres posibles causas.**

10. **Escribe en pseudocódigo un programa que encienda un LED si la lectura de una LDR es menor que `300` y lo apague si no.**

Practica

1. **En un diagrama de flujo, ¿qué figura representa una decisión?**
   - **A)** Rectángulo
   - **B)** Rombo
   - **C)** Óvalo
2. **¿Qué es una variable?**
   - **A)** Un error del programa
   - **B)** Un espacio con nombre donde se guarda un dato
   - **C)** Un tipo de sensor
3. **¿Qué estructura usarías para repetir algo 20 veces?**
   - **A)** if
   - **B)** for
   - **C)** else
4. **En Arduino, ¿qué función se ejecuta una sola vez al encender la placa?**
   - **A)** loop()
   - **B)** setup()
   - **C)** delay()
5. **¿Entre qué valores da la lectura analogRead() en Arduino UNO?**
   - **A)** 0 y 1
   - **B)** 0 y 1023
   - **C)** 0 y 100
6. **En programación, == significa...**
   - **A)** asignar
   - **B)** comparar si son iguales
   - **C)** sumar
7. **10 % 3 vale...**
   - **A)** 1
   - **B)** 3
   - **C)** 3,33
8. **El operador && es verdadero si...**
   - **A)** se cumple al menos una condición

B) se cumplen las dos

C) no se cumple ninguna

9. **Un bucle que nunca termina se llama...**

   A) bucle infinito

   B) función

   C) variable

10. **Para ver el valor de una variable desde la placa se usa...**

    A) el monitor serie

    B) un servomotor

    C) un relé

Explícalo con tus palabras

Incluye: Qué es un algoritmo, con un ejemplo de la vida diaria; Los símbolos de un diagrama de flujo; Para qué sirven las variables y sus tipos; La diferencia entre un condicional y un bucle; Cómo lee un sensor y controla un LED una placa Arduino.

Resumen



Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

¿Qué es un algoritmo?

Una secuencia ordenada y finita de pasos para resolver un problema.

Símbolo de decisión en un diagrama de flujo

El rombo, con salidas sí y no.

Diferencia entre for y while

for repite un número fijo de veces; while repite mientras se cumpla una condición.

Funciones obligatorias en Arduino

setup() (una vez) y loop() (se repite siempre).

¿Qué hace analogRead()?

Lee un sensor analógico y devuelve un valor entre 0 y 1023.

Diferencia entre = y ==

= asigna un valor; == compara.

Bucle while

Repite mientras se cumpla una condición.

Monitor serie

Ventana donde la placa envía datos al ordenador; útil para depurar.

Soluciones

1. **B)** El rombo tiene dos salidas: sí y no.
2. **B)** Su valor puede cambiar mientras el programa se ejecuta.
3. **B)** El bucle for repite un número fijo de veces.
4. **B)** setup() configura los pines; loop() se repite sin fin.
5. **B)** El conversor de 10 bits da 1024 valores posibles.
6. **B)** Un solo = asigna.
7. **A)** Es el resto de 10 / 3.
8. **B)** Es el Y lógico.
9. **A)** Su condición nunca es falsa.
10. **A)** Serial.println(valor).

Soluciones de los ejercicios

1. int, float, bool (boolean) y String.
2. 17 % 5 = 2; 20 % 4 = 0; 9 / 2 con enteros = 4 (se pierde la parte decimal).
3. Falsa: la primera es verdadera pero la segunda no, y && exige las dos.
4. 8 veces (i = 0 a 7).
5. for (int i = 10; i >= 1; i--) { Serial.println(i); }
6. for se usa cuando se sabe cuántas veces repetir; while repite mientras se cumpla una condición, sin saber de antemano cuántas veces.
7. setup() se ejecuta una vez al principio para configurar; loop() se repite indefinidamente con el programa principal.
8. int doble(int n) { return n * 2; }
9. El LED está conectado al revés, el número de pin del programa no coincide con el del circuito o falta configurar el pin como salida (también un cable suelto).
10. loop: luz = leer A0; si luz < 300 → LED encendido; si no → LED apagado.